

# Load Balancing Simulator

## Project Proposal

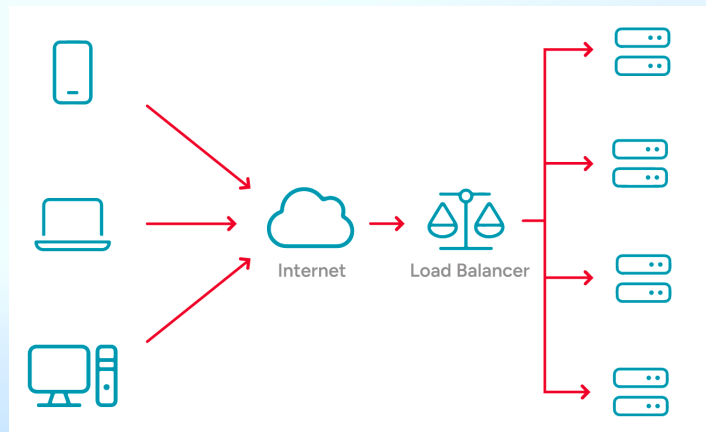
Daniel Thomas, Kimone Walker,  
Neev Jain, Sean O'Connor

# Background and Purpose

Changing a load-balancing policy or increasing traffic can overload some servers while leaving others underused. Testing these changes in production can affect real users.

For our project, we are setting out to build a load-balancing simulator, which allows us to test different system layouts and distribution policies without paying for server use, or affecting production deployments.

The main question we set to answer is:  
*what changes when we send the same workload through a different policy, or increase the traffic?*



# Goals and Requirements

## 1. The user must be able to configure a simulation

- Concrete server parameters (size, count, concurrency limit, etc)

## 1. The system generates repeatable, predictable traffic

- Saves random seed, experiments can be repeated

## 1. The user can compare load-balancing policies

- One run can use round-robin, another round using least-active-requests

## 1. The user must be able to track the full request lifecycle

- An action can be followed from queuing through execution

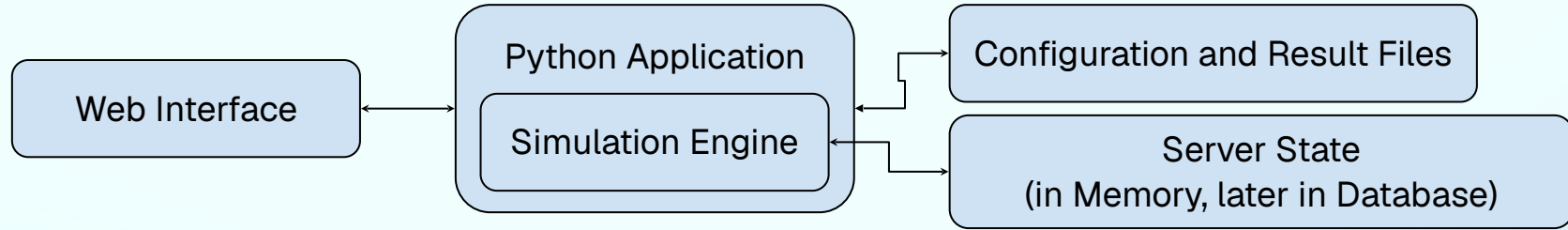
## 1. The system must show useful, actionable results

- Metrics include average response time, utilization, failure rate
- Comparisons can be made between runs

## 1. The system must make experiments reproducible

- Given the same configuration, a repeated run must result in the same output

# Architecture and Implementation



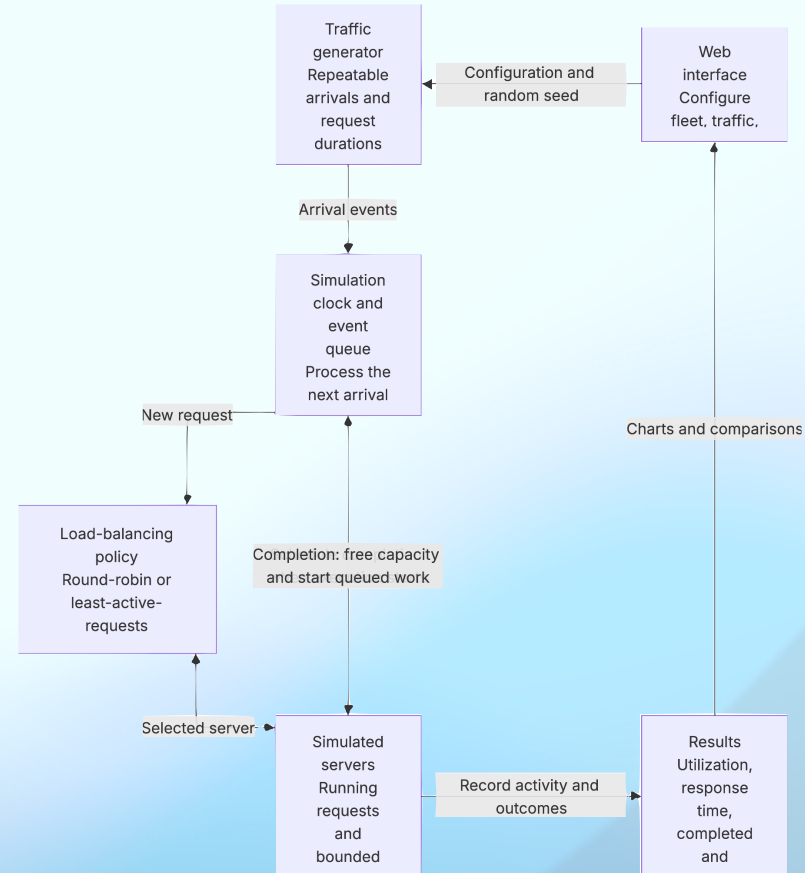
Our application is comprised of a few layers:

- Web Interface: Displays controls and results, allows control of simulation
- Python Application: Handles communication between layers, receives configurations and invokes simulation engine
  - Simulation Engine: Given parameters, outputs how a system should appear at a given time

# Simulation Engine Architecture

- **Traffic generator:** Creates repeatable request arrivals and service durations using a saved random seed.
- **Event engine:** Advances simulated time to the next arrival or completion.
- **Load balancer:** Chooses a server using round-robin or least-active-requests.
- **Server model:** Tracks running requests and waiting queues; completions free capacity, and full queues reject requests.
- **Metrics collector:** Records utilization, response times, completions, and rejections.

**Key idea:** Every request changes server state, which informs the next routing decision.



# Sample User Workflow

- The user begins by setting configurations (server count, server size, req/s, request/duration range, simulation duration, server concurrency limit, and queue limit)
- The system then runs the simulation, recording per-server utilization, a list of completed and rejected requests, and average and p95 response time in the process
  - If the assigned server is busy, the request will be placed in a queue. If the queue is full, reject the request
  - When a request in a given server is completed, it starts the next request in the queue
- The system gives the option to save configuration results and/or compare with other saved results

# Advanced Goals

Our previously-defined goals give enough for a working project, but more advanced goals may be attempted if time and resources allow. These include, but are not limited to:

- Server Types (different architectures, different core counts)
- More Policies (capacity weighted routing, least resource use)
- Regions and network delay (introducing latency between server groups)
- Failures and stale information (unavailable servers, delayed load updates)
- Database-backed state (allowing for saving, periodic re-execution, and scheduling of simulations)
- Cloud deployment and containers (the system can be available publicly, instead of running on our machinery)
- Larger experiments (pushing the boundaries of our simulation runtime)

# Measuring Completeness

Our system will be evaluated in three ways:

- *Correctness*: Check small examples by hand, including one server, two servers, simultaneous arrivals/completions, and a full queue. After a run finishes, every request must be completed or rejected, with no occupied slots left over.
- *Repeatability*: The same configuration, seed, and policy must produce the same simulated results.
- *Policy comparison*: Run both policies on the same workloads at low, near-capacity, and overloaded traffic levels, plus a scenario with 20% more traffic. Repeat each scenario with three seeds and report variation.



# Milestones

| <b>Demo 1 ✓<br/>(Sep 23)</b>                                                                                                        | <b>Demo 2<br/>(Oct 21)</b>                                                                                                                      | <b>Demo 3<br/>(Nov 16)</b>                                                                                                                                                                      | <b>Final<br/>(Dec 9)</b>                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- Proposal</li><li>- Architecture</li><li>- Problem scope</li><li>- Evaluation plan</li></ul> | <ul style="list-style-type: none"><li>- Working simulator</li><li>- One policy</li><li>- Basic interface</li><li>- Correctness checks</li></ul> | <ul style="list-style-type: none"><li>- Two-policy comparison</li><li>- Saved experiment support</li><li>- Measured improvement over Demo 2 (performance, repeatability, correctness)</li></ul> | <ul style="list-style-type: none"><li>- Finished application</li><li>- Reproducible experiments</li><li>- Documentation and help systems for new users</li></ul> |

Thank you.

# References

- Meta-provided paper on load-balancing technologies: <https://www.usenix.org/conference/osdi23/presentation/saokar>
- Class project proposal  
[https://ec528.github.io/\\_pages/projects/EC528-fall26-load-balancing.pdf](https://ec528.github.io/_pages/projects/EC528-fall26-load-balancing.pdf)
- Cloudflare brief on load balancing concepts  
<https://www.cloudflare.com/learning/performance/what-is-load-balancing/>
- Zoom meeting with mentor Shripad Nadgowda, Engineering Manager at Meta
- Class notes